

The Cost of Unlinkability: An Empirical Evaluation of A²L Anonymous Atomic Swaps on Bitcoin Taproot

Yan Fernandes
UCSal
Salvado, Brazil
yanferpi@proton.me

Abstract

Submarine swaps used by production Bitcoin services (Boltz, Loop) rely on Hash Time-Locked Contracts (HTLCs) whose preimage hash is identical on both legs of a swap, allowing trivial linking of sender and receiver by any observer. *Anonymous Atomic Locks* (A²L, Tairi et al., IEEE S&P 2021) remove this correlation by replacing the shared hash with Schnorr adaptor signatures locked to randomizable Castagnos–Laguillaumie (CL) puzzles. While the cryptographic security of A²L is well established, the *engineering cost* of adopting it in production-grade Bitcoin software remains unquantified. We present a controlled, cross-architecture experiment comparing a Rust reference implementation of A²L on Bitcoin Taproot (TORTUGA) against an HTLC baseline. We measure end-to-end latency, on-chain footprint, fees, and peak memory across $N = 30$ runs per arm on two heterogeneous machines (Apple M4 Pro/ARM64 and AMD EPYC-Genoa/x86_64). [Results placeholder: report effect sizes for H1–H4 with 95 % CIs.] Our contribution is the first empirical characterisation of the unlinkability–cost trade-off for A²L on Bitcoin and a fully replicable benchmarking harness for future privacy-preserving swap research.

Keywords

Empirical software engineering, atomic swaps, privacy, Bitcoin, adaptor signatures, replication study

1 Introduction

Bitcoin submarine swaps—the operational backbone of off-chain liquidity services such as Boltz and Lightning Loop—use HTLCs in which the same SHA-256 preimage hash H appears on both legs of the swap. Although each leg is a separate transaction, any observer can match the two by inspecting H , breaking unlinkability between payer and payee. The privacy of the user therefore depends entirely on the trustworthiness of the swap provider.

The Anonymous Atomic Locks construction (A²L) of tairi2021a2l removes this trust assumption. A²L replaces the shared hash with a pair of *randomized adaptor signatures* bound to a CL-encrypted puzzle; each leg of the swap commits to a different adaptor point on-chain, so the two transactions are cryptographically unlinkable even by the provider.

While A²L’s cryptographic guarantees have been proven, its *engineering cost* in a production-grade Bitcoin setting (Taproot, BIP-340 Schnorr) has not been empirically characterised. Practitioners need quantitative answers to deploy this privacy improvement.

Research Questions.

- **RQ1** What is the end-to-end latency overhead of A²L compared to HTLC submarine swaps?
- **RQ2** What is the on-chain footprint difference (vbytes, fees in sat/vB) between A²L and HTLC?
- **RQ3** What is the peak memory and per-phase computational overhead of A²L relative to HTLC?

Contributions. (i) TORTUGA, an open-source Rust implementation of A²L on Bitcoin Taproot (5 crates, 82 tests). (ii) A pre-registered, controlled, cross-architecture experiment comparing A²L against an HTLC baseline. (iii) A fully replicable benchmarking harness (instrumented CSV emission, analysis pipeline, Dockerfile, frozen commit) archived on Zenodo.

2 Background

2.1 HTLC Submarine Swaps

A submarine swap exchanges off-chain liquidity (e.g., Lightning) for on-chain funds. Both legs are funded with the script `IF <hash> EQUALVERIFY <pubA> CHECKSIG ELSE <timeout> CLTV DROP <pubB> CHECKSIG ENDIF`. The recipient learns the preimage r where $\text{SHA256}(r) = H$ to claim funds; revealing r unlocks the other leg. The shared H makes the two transactions trivially linkable.

2.2 Adaptor Signatures and CL Puzzles

A Schnorr adaptor signature is a pre-signature $\tilde{\sigma}$ over an adaptor point $T = t \cdot G$; revealing the full signature σ discloses t . A²L composes adaptor signatures with a Castagnos–Laguillaumie public-key encryption scheme whose ciphertexts form a *randomizable puzzle*: anyone with the puzzle and a fresh randomizer ρ can produce a new puzzle Z' for $t + \rho$ without knowledge of t . The unlinkability of A²L follows from puzzle randomizability and Schnorr adaptor security tairi2021a2l, aumayr2021generalized.

3 Related Work

Privacy-preserving payment hubs. TumbleBit heilman2017tumblebit introduced unlinkable hub payments on Bitcoin using RSA puzzles; A²L tairi2021a2l reduces the cryptographic and on-chain footprint and is the first construction compatible with Taproot key-path spends. BlindHub qin2023blindhub extends A²L with blind signatures.

Empirical evaluation of cryptographic protocols. Performance studies of payment-channel protocols harris2020flare, roos2018settling measure throughput and latency in simulated networks; we focus instead on *per-swap* cost in a real Bitcoin regtest network with side-by-side baseline.

Table 1: Operationalisation of dependent variables.

Variable	Unit	Source
e2e_latency_ms	ms	wall-clock init→claim
pgen_ms	ms	cl-crypto (A ² L)
prand_ms	ms	cl-crypto (A ² L)
psolve_ms	ms	cl-crypto (A ² L)
cldl_prove_ms	ms	ZK proof (A ² L)
cldl_verify_ms	ms	ZK proof (A ² L)
adaptor*_ms	ms	adaptor sign/verify/complete
htlc*_ms	ms	HTLC build/claim
tx_vbytes	vB	rust-bitcoin vsize()
fee_sats	sat	vbytes × 5 sat/
peak_rss_kb	kB	getrusage (normalised)
cpu_user/sys_ms	ms	getrusage

Table 2: Pre-registered hypotheses. All tests are two-sided unless noted.

ID	Alternative hypothesis H _i
H1	median end-to-end latency of A ² L is greater than HTLC
H2	median on-chain vbytes of A ² L is smaller than HTLC
H3	median fee in satoshis of A ² L is smaller than HTLC
H4	median peak RSS of A ² L is greater than HTLC

Experimental software engineering on cryptographic systems. Replication studies remain rare in the cryptographic-systems space wohlin2014guidelines,carver2010replications; our work follows the guidelines of wohlin2012experimentation for design, randomisation, and threats-to-validity reporting.

4 Methodology

We follow a single-factor, two-level, between-subjects controlled experiment. The protocol is pre-registered: table 3 summarises the design and table 1 the operationalisation.

4.1 Object of Study

TORTUGA is a Rust workspace of five crates (cl-crypto, adaptor, protocol, bitcoin, cli) totalling 82 unit and integration tests. It exposes a compare subcommand that runs A²L and HTLC swaps side-by-side against a Nigiri Bitcoin regtest node.

4.2 Independent Variable

PROTOCOL $\in \{A^2L, HTLC\}$.

4.3 Dependent Variables

Table 1 lists the 12 dependent variables with units and source.

4.4 Hypotheses

Table 2 states the four pre-registered hypotheses. We use a significance level $\alpha = 0.05$ with Bonferroni correction across the four families ($\alpha' \approx 0.0125$).

Table 3: Experimental design summary.

Element	Choice
Design	single-factor, between-subjects
Factor	Protocol {A ² L, HTLC}
Sample size	$N = 30$ per arm (60 total) per machine
Randomisation	run order randomised; container reset/run
Warm-up	3 discarded runs per arm
Hardware (M1)	Apple M4 Pro, 24 GB, macOS
Hardware (M2)	AMD EPYC-Genoa 8 vCPU, 15 GB, NixOS 24.11
Toolchain	rustc 1.82+ -release, LTO, mimalloc
Network	Nigiri regtest, fresh container/run
Swap amount	100 000 sat
Fee rate	5 sat/
CL security	1827-bit class-group discriminant

4.5 Experimental Design

4.6 Cross-Architecture Replication

We replicate the full design on two heterogeneous machines: M1 (Apple M4 Pro, ARM64, macOS) and M2 (AMD EPYC-Genoa, x86_64, NixOS 24.11, kernel 6.6.94). This provides an external-validity probe against architecture-specific SIMD/aes-ni effects and OS-specific scheduler noise. getrusage.ru_maxrss units differ between Linux (kB) and macOS (bytes); the harness normalises to kB before emission.

4.7 Statistical Analysis Plan

For each dependent variable: (i) Shapiro–Wilk normality test; (ii) Levene’s test for variance equality; (iii) if normal and equal variance, Welch’s t -test, else Mann–Whitney U ; (iv) effect size as Cohen’s d (parametric) or Cliff’s δ (non-parametric) with 95 % bootstrap CI (10 000 resamples); (v) Bonferroni correction across H1–H4. All analyses pre-registered in analysis/run.py (deliverable #4).

5 Results

[Placeholder]. Descriptive statistics (median, IQR, mean±SD) per arm per machine. See tables/descriptives.tex.

[Placeholder]. Hypothesis-test outcomes for H1–H4 with effect sizes and 95 % CIs. See tables/hypotheses.tex.

[Placeholder]. Boxplots of per-phase timings. See figures/timings.pdf.

[Placeholder]. Cross-architecture comparison (M1 vs M2). See figures/cross_arch.pdf.

6 Discussion

[Placeholder]. Interpretation of effect sizes in operational terms (e.g., how many additional ms per swap; sat per swap saved by Taproot keypath spends).

[Placeholder]. Implications for production swap providers (Boltz/Loop): is the latency overhead amortisable across a payment hub? Is the on-chain saving enough to subsidise the CL cost?

7 Threats to Validity

We follow the taxonomy of wohlin2012experimentation.

Construct. Our metrics (latency, vbytes, fees, RSS) directly operationalise “engineering cost”, but developer-experience cost (LOC, complexity, on-call burden) is out of scope for this short paper.

Internal. JIT/warm-cache effects mitigated by 3 warm-up runs per arm; background load mitigated by a systemd-run cgroup (M2) and exclusive use (M1); allocator pinned (mimalloc); run order randomised within blocks of 10.

External. Generalisation to mainnet block latency is limited by regtest’s artificial block intervals; we report *protocol-internal* latency separately from on-chain confirmation latency. TORTUGA is the only A²L implementation benchmarked; cross-architecture replication mitigates platform-specific generalisation concerns.

Conclusion. $N = 30$ per arm gives power ≥ 0.80 for Cliff’s $\delta \geq 0.33$ (post-hoc verification). Bonferroni correction is conservative but appropriate. Replication package frozen at commit <hash> and published on Zenodo.

8 Conclusion and Future Work

We present the first empirical evaluation of A²L on Bitcoin Taproot and quantify the unlinkability–cost trade-off relative to HTLC submarine swaps. [Results placeholder: 1–2 sentence headline finding once data is in.] Future work includes (i) developer-experience metrics on TORTUGA; (ii) end-to-end measurements on mainnet/testnet with realistic block latency; (iii) benchmarking alternative CL backends.

Replication Package. Source, raw CSVs, analysis scripts, and Dockerfile are available at <https://github.com/yan-pi/tortuga-swap> (tag `esem20XX-replication`) and archived on Zenodo (DOI on submission).

References